

SOFTWARE DESIGN SPECIFICATION
for
Mahindra University Mobile Application

iOS & Android — Flutter + Firebase

Version 1.0

Document Information

Title:	Software Design Specification — Mahindra University Mobile App
Document Version No:	1.0
Document Version Date:	March 2026
Prepared By:	Group 58
Preparation Date:	March 2026

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename
1.0	March 2026	Srishanth Reddy - SE23UCSE084 Laasya Reddy - SE23UCSE149	SDS	MU_App_SDS

1 Introduction

This Software Design Specification (SDS) document describes the complete software design for the Mahindra University Mobile Application (MU App) — a cross-platform iOS and Android application built using Flutter, Firebase, and WebView integration. This document translates the requirements captured in the SRS into a concrete design, covering architecture, component structure, data models, exception handling, and quality of service.

1.1 Purpose

This SDS defines the architectural and detailed design of the MU App for the development team, project stakeholders, academic evaluators, and the MU IT department. It describes how the system is structured, how components interact, and how the design satisfies the functional and non-functional requirements. Readers are expected to use this document as the primary reference during implementation, code review, and testing.

1.2 Scope

This SDS applies to the complete MU App system including the Flutter mobile client, Firebase backend (Auth, Firestore, Cloud Messaging), and WebView integrations with the MU ERP portal and MU website. It covers the Login/Register flow, Home Dashboard, Notices Board, ERP Portal, MU Website viewer, and Profile screens. It does not cover the MU ERP server internals, Moodle LMS internals, or the university's existing WordPress website backend.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
MU App	The Mahindra University Mobile Application described in this document
Flutter	Google's open-source UI framework for building natively compiled apps from a single Dart codebase
Firebase	Google's Backend-as-a-Service (BaaS) platform providing Auth, Firestore, and FCM
Firestore	Firebase's NoSQL cloud database used for notices, events, and user profiles
FCM	Firebase Cloud Messaging — push notification delivery service
ERP	Enterprise Resource Planning portal at muerp.mahindrauniversity.edu.in
WebView	A browser embedded within the Flutter app to display web content
Provider	Flutter state management pattern using ChangeNotifier
GoRouter	Flutter navigation package used for declarative routing
RBAC	Role-Based Access Control — permissions based on user role (student/faculty/parent)
SDS	Software Design Specification — this document
SRS	Software Requirements Specification — the requirements document
Dart	The programming language used with Flutter

APK	Android Package — the installable Android app binary
------------	--

1.4 References

- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions
- Mahindra University SRS v1.0 — Software Requirements Specification for MU App
- Flutter Documentation — <https://docs.flutter.dev>
- Firebase Documentation — <https://firebase.google.com/docs>
- go_router Package — https://pub.dev/packages/go_router
- webview_flutter Package — https://pub.dev/packages/webview_flutter
- provider Package — <https://pub.dev/packages/provider>
- Mahindra University Official Website — <https://www.mahindrauniversity.edu.in>

2 Use Case View

The MU App serves four primary actor types: Student, Faculty, Parent, and Visitor. The following use cases represent the central functionality of the system and drive the major design decisions in this document.

2.1 Use Cases

ID	Use Case Name	Brief Description
UC1	User Login	Student, Faculty, or Parent authenticates using email and password via Firebase Auth. On success, routed to role-specific Home dashboard.
UC2	User Registration	New user registers with name, email, password, and role. Firebase Auth account and Firestore profile document are created simultaneously.
UC3	View Home Dashboard	Authenticated user views personalized dashboard showing greeting, quick-access cards (ERP, MU Website, Notices), and recent notices fetched from Firestore.
UC4	View Notices	Student views notices filtered by their class. Faculty views all notices. Notices are fetched from Firestore ordered by timestamp descending.
UC5	Post Notice	Faculty user composes a notice with title, body, and category, and publishes it to Firestore. All relevant students see it in real time.
UC6	Access ERP Portal	Authenticated user opens the MU ERP portal (muerp.mahindrauniversity.edu.in) embedded inside the app via WebView with mobile CSS injection.
UC7	Browse MU Website	Any user (including visitors) browses the official MU website embedded in the app via WebView.
UC8	View Profile & Logout	Authenticated user views their name, email, role, and MU contact info. User logs out, terminating the Firebase session and redirecting to Login.

2.2 Use Case Detail — UC1: User Login

Use Case ID	UC1
Use Case Name	User Login
Actors	Student, Faculty, Parent
Preconditions	User has a registered account. App is installed and launched.

Main Flow	1. User enters email and password on LoginScreen. 2. App calls AuthProvider.login(). 3. Firebase Auth validates credentials. 4. AuthProvider fetches Firestore user document. 5. GoRouter redirects to /home.
Alternative Flow	Wrong credentials: FirebaseAuthException returned. Error message displayed in red container. User retries.
Postconditions	User is authenticated. Session persists across app restarts. FCM token saved.

2.3 Use Case Detail — UC5: Faculty Posts Notice

Use Case ID	UC5
Use Case Name	Faculty Posts Notice
Actors	Faculty (trigger)
Preconditions	User is logged in with role = faculty. Firestore notices collection exists.
Main Flow	1. Faculty navigates to Notices tab. 2. Taps red FloatingActionButton (only visible to faculty). 3. Modal bottom sheet opens with Title, Body, Category fields. 4. Faculty fills fields and taps Publish. 5. NoticesProvider.post() creates Firestore document. 6. Students see notice immediately on next fetch.
Alternative Flow	Empty fields: Firestore write still proceeds. Title/body validation recommended in future release.
Postconditions	Notice document created in Firestore. Visible to all relevant students immediately.

3 Design Overview

The MU App follows a layered architecture with clear separation between UI, state management, services, and backend. The design is guided by Flutter best practices using the Provider pattern for state management and GoRouter for navigation. The backend is entirely Firebase-based, eliminating the need for a custom server.

3.1 Design Goals and Constraints

- Single codebase: Flutter must produce identical behavior on Android (API 24+) and iOS (iOS 12+)
- Backend: Firebase Spark (free) plan must be sufficient for MVP — no custom server infrastructure
- ERP/Moodle: Must use WebView embedding — no official API is exposed by MU for these portals
- No AI: OpenAI or any paid AI service is explicitly excluded from this design
- State management: Provider pattern is mandatory — no Riverpod, BLoC, or other alternatives
- Navigation: GoRouter with redirect guards for authentication enforcement
- Security: No API keys in client code. Firestore security rules enforce RBAC server-side
- Offline: Firestore local persistence enabled for basic offline read capability
- Performance: App cold start must be under 3 seconds on 4G; notice fetch under 2 seconds

3.2 Design Assumptions

- MU ERP portal at muerp.mahindrauniversity.edu.in is accessible over HTTPS and renders in WebView
- MU website at mahindrauniversity.edu.in is publicly accessible without authentication
- Users have Android or iOS smartphones with internet access (3G minimum)
- Firebase services remain operational with Spark plan limits not exceeded during MVP phase
- All user data (name, email, role) is voluntarily provided during registration — no integration with MU's existing student database in MVP
- Faculty role is self-declared during registration. Role verification by MU admin is a future enhancement

3.3 Significant Design Packages

The system is decomposed into the following packages:

Package	Responsibility
lib/screens/	All UI screens: auth, home, notices, erp, moodle, profile. Each screen is a StatefulWidget or StatelessWidget.
lib/widgets/	Reusable UI components: AppShell (bottom nav), and future shared widgets.

lib/providers/	State management layer: AuthProvider (authentication state), NoticesProvider (notices data). All extend ChangeNotifier.
lib/models/	Data models: UserModel, NoticeModel. Define data shapes and Firestore serialization/deserialization.
lib/core/constants/	Design system: AppColors (Mahindra Red palette), shared styling constants.
lib/core/services/	Service layer for future integrations (AI, push notifications). Currently minimal.
lib/router.dart	GoRouter configuration: route definitions, auth redirect guard, ShellRoute for bottom nav.
lib/main.dart	Entry point: Firebase initialization, Provider tree setup, MaterialApp.router configuration.

3.4 Dependent External Interfaces

The following external interfaces are required by the MU App:

External Application & Interface	Module Using the Interface	Functionality / Description
Firebase Authentication firebase_auth SDK	AuthProvider	signInWithEmailAndPassword(), createUserWithEmailAndPassword(), authStateChanges() stream, signOut(). Manages session persistence.
Cloud Firestore cloud_firestore SDK	AuthProvider, NoticesProvider	Collection reads (users, notices), document writes, orderBy queries. Provides real-time data sync.
Firebase Cloud Messaging firebase_messaging SDK	Future NotificationService	Push notification delivery for new notices. FCM token retrieval and storage.
MU ERP Portal HTTPS WebView	ERPScreen	muerp.mahindrauniversity.edu.in loaded in WebViewController. JavaScript injection for mobile CSS.
MU Website HTTPS WebView	MoodleScreen (renamed)	www.mahindrauniversity.edu.in loaded in WebViewController. No authentication required.
Android/iOS OS url_launcher	ProfileScreen (future)	Deep links for tel:, mailto:, and maps: URLs for campus contact actions.

3.5 Implemented Application External Interfaces

The MU App exposes no public interfaces to external systems in the MVP release. All data flows are inbound (the app consumes Firebase and WebView services). Future releases may expose a REST API for admin tooling.

Interface Name	Module Implementing	Description
None in MVP	N/A	No external interfaces published by MU App in v1.0

4 Logical View

This section describes the detailed design of the MU App. It presents the class design model showing key classes and their relationships, followed by use case realizations showing how design components collaborate to implement each major use case.

4.1 Design Model

The MU App is structured around the Provider pattern. The following classes form the core of the design:

Class	Layer	Responsibilities & Key Operations
AuthProvider	Provider (State)	Manages authentication state. Listens to Firebase Auth stream. Exposes: login(email, password), register(name, email, password, role), logout(), currentUser, isLoggedIn, isLoading, role.
NoticesProvider	Provider (State)	Manages notices data. Exposes: fetch(studentClass), post(notice), notices list, isLoading, error. Reads/ writes Firestore notices collection.
UserModel	Model	Defines user data shape. Fields: uid, name, email, role, department, studentClass. Methods: fromFirestore(), toFirestore().
NoticeModel	Model	Defines notice data shape. Fields: id, title, body, category, author, createdAt, targetClass, isPinned. Methods: fromFirestore(), toFirestore().
AppColors	Constants	Centralizes Mahindra brand colors: primary (#C8102E), primaryDark, background, surface, textPrimary, textSecondary, border, error, urgent, academic, event, general, moodle.
LoginScreen	Screen (UI)	Stateful widget. Contains TextEditingControllers for email/password, Form with validation, calls AuthProvider.login(). Shows loading spinner and error container.
RegisterScreen	Screen (UI)	Stateful widget. Collects name, email, password, role. Calls AuthProvider.register(). On success navigates to /home.
HomeScreen	Screen (UI)	Stateful widget. Reads AuthProvider and NoticesProvider. Renders SliverAppBar with red gradient, 3 quick-access cards, recent notices list.
NoticesScreen	Screen (UI)	Stateful widget. Reads NoticesProvider. Shows ListView of notice cards with color-coded left border. FAB shown only for faculty role.

ERPScreen	Screen (UI)	Stateful widget. Contains WebViewController loading ERP URL. Shows LinearProgressIndicator during load. Injects mobile CSS on page finish.
MoodleScreen	Screen (UI)	Stateful widget. Contains WebViewController loading MU website. Same pattern as ERPScreen.
ProfileScreen	Screen (UI)	Stateless widget. Reads AuthProvider. Shows user info, MU contacts, Logout button.
AppShell	Widget	StatelessWidget wrapping ShellRoute children. Contains NavigationBar with 3 tabs: Home, Notices, Profile. Reads current route from GoRouterState.
appRouter	Router	GoRouter instance. Defines all routes. Contains redirect guard: unauthenticated → /login, authenticated on auth page → /home.

4.2 Use Case Realization

4.2.1 UC1: User Login — Sequence

The following describes the interaction sequence for the User Login use case:

1. User opens app. GoRouter checks AuthProvider.isLoggedIn.
2. isLoggedIn is false → router redirects to /login. LoginScreen renders.
3. User enters email and password. Taps Sign In button.
4. LoginScreen calls context.read<AuthProvider>().login(email, password).
5. AuthProvider calls FirebaseAuth.signInWithEmailAndPassword(email, password).
6. Firebase Auth validates credentials. Returns UserCredential on success or FirebaseAuthException on failure.
7. On success: authStateChanges() stream fires. AuthProvider fetches Firestore users/{uid} document.
8. UserModel is constructed from Firestore data. _user is set. notifyListeners() called.
9. GoRouter redirect fires again. isLoggedIn is now true. Routes to /home.
10. HomeScreen builds with personalized greeting from AuthProvider.currentUser.

4.2.2 UC5: Faculty Posts Notice — Sequence

11. Faculty navigates to /notices tab. NoticesScreen builds.
12. NoticesScreen reads AuthProvider.role. Role is 'faculty' → FloatingActionButton rendered.
13. Faculty taps FAB. _showPost(context) called. showModalBottomSheet opens.
14. Faculty fills title, body, selects category from dropdown. Taps Publish.
15. NoticeModel created with faculty's name from AuthProvider.currentUser.name.
16. NoticesProvider.post(notice) called. Calls
Firestore.collection('notices').add(notice.toFirestore()).
17. Firestore write succeeds. NoticesProvider.fetch() called to refresh list.
18. Navigator.pop() closes the bottom sheet. Notices list re-renders with new notice.

4.2.3 UC6: Access ERP Portal — Sequence

19. User taps ERP Portal card on HomeScreen. `context.push('/erp')` called.
20. GoRouter navigates to ERPScreen (outside ShellRoute, no bottom nav).
21. ERPScreen.initState() creates WebViewController.
22. WebViewController.loadRequest() called with `muerp.mahindrauniversity.edu.in/login.htm`.
23. NavigationDelegate.onProgress fires. LinearProgressIndicator value updates.
24. NavigationDelegate.onPageFinished fires. `_loading` set to false. Progress bar hidden.
25. JavaScript injected via `runJavaScript()` to set viewport meta tag for mobile rendering.
26. User logs in on ERP portal using their university credentials (handled by ERP system).

5 Data View

The MU App uses Firebase Firestore as its persistent data store. Data is organized into collections. This section describes the domain model, data model, and data dictionary.

5.1 Domain Model

The core domain entities and their relationships are:

- User — a registered person with a role. One User can post many Notices (if faculty). One User has one Profile.
- Notice — an announcement posted by a faculty member. Belongs to one User (author). May target a specific class or all students.
- Session — managed by Firebase Auth. Not persisted in Firestore. Auto-renewed by Firebase SDK.
- ERPSession — managed by the ERP WebView cookie store. Independent of Firebase auth.

Relationships: User 1..* posts Notice (if faculty). Notice 0..1 targets StudentClass. User * belongs to 1 Department.

5.2 Data Model (Persistent Data View)

Firestore is a NoSQL document database. Collections and their document structures:

users collection

Field	Type	Required	Description
uid (doc ID)	String	Yes	Firebase Auth UID. Auto-generated. Primary key.
name	String	Yes	Full name entered during registration. Max 100 chars.
email	String	Yes	Email address. Must be valid format. Unique in Firebase Auth.
role	String (enum)	Yes	Values: 'student', 'faculty', 'parent', 'admin'. Controls navigation and Firestore write permissions.
department	String?	No	MU school: e.g., 'ECSoE', 'Management'. Null for parents.
studentClass	String?	No	Class code: e.g., 'CS-3A'. Used for notice filtering. Null for faculty.
createdAt	Timestamp	Yes	Server timestamp of account creation. Set by <code>FieldValue.serverTimestamp()</code> .

notices collection

Field	Type	Required	Description
id (doc ID)	String	Yes	Auto-generated by Firestore. Used as NoticeModel.id.
title	String	Yes	Notice headline. Displayed in card and list.
body	String	Yes	Full notice text. Max 5000 chars.
category	String (enum)	Yes	Values: 'general', 'urgent', 'academic', 'event'. Controls card color.
author	String	Yes	Display name of posting user. Copied from UserModel.name.
createdAt	Timestamp	Yes	Server timestamp. Used for ordering (descending).
targetClass	String?	No	Target class code. Null = all students. 'CS-3A' = specific class.
isPinned	Boolean	No	If true, notice appears at top of list. Default: false.

5.2.1 Data Dictionary

Variable	Entity	Type	Constraints & Notes
_user	AuthProvider	UserModel ?	Null when logged out. Set by Firestore fetch after Firebase Auth success.
_loading	AuthProvider	bool	True during login/register async calls. Controls button disabled state and spinner.
_notices	NoticesProvider	List<Notice Model>	Ordered by createdAt descending. Filtered by studentClass if provided.
_error	NoticesProvider	String?	Set on Firestore read failure. Displayed in error state UI.
primary	AppColors	Color	#C8102E — Mahindra Red. Used for AppBar, buttons, active nav icon, ERP header.
moodle	AppColors	Color	#7C3AED — Purple. Used for MU Website card and screen AppBar.
role	UserModel	String	Enum: 'student' 'faculty' 'parent' 'admin'. Read by router and NoticesScreen for conditional rendering.
targetClass	NoticeModel	String?	Null = global notice. Set = class-specific. Compared against user.studentClass during fetch filtering.

6 Exception Handling

The MU App handles exceptions at two levels: Firebase/network exceptions caught in Providers, and UI-level error states displayed to users.

Exception	Where Thrown	Handling	User-Facing Response
FirebaseAuthException (user-not-found)	AuthProvider.login()	Caught in try-catch. Returns error string.	Red error container: 'No account with this email.'
FirebaseAuthException (wrong-password)	AuthProvider.login()	Caught in try-catch. Returns error string.	Red error container: 'Incorrect password.'
FirebaseAuthException (email-already-in-use)	AuthProvider.register()	Caught in try-catch. Returns error string.	Red error container: 'Email already registered.'
Firestore read failure	NoticesProvider.fetch()	Caught in try-catch. Sets _error string.	Error text shown in body: 'Failed to load notices.'
Firestore write failure	NoticesProvider.post()	Caught in try-catch. Returns false.	Bottom sheet stays open. No success message shown.
WebView load failure	ERPScreen, MoodleScreen	NavigationDelegate.onWebResourceError	Progress bar hidden. User sees partial or blank WebView. Refresh button available.
Network timeout	All HTTP/Firebase calls	Firebase SDK handles with built-in retry.	Loading indicators persist. User can pull- to-refresh.
Null user after logout	ProfileScreen role badge	Null-safe operator ?.isEmpty check	Empty string rendered instead of crash.

7 Configurable Parameters

The following parameters control the behavior of the MU App and can be modified without a full rebuild in future releases.

Configuration Parameter Name	Definition and Usage	Dynamic?
ERP_URL	URL loaded by ERPScreen WebView. Currently: https://muerp.mahindrauniversity.edu.in/login.htm . Can be updated if MU changes the ERP URL.	No
MU_WEBSITE_URL	URL loaded by MoodleScreen WebView. Currently: https://www.mahindrauniversity.edu.in . Updated if MU website URL changes.	No
NOTICES_FETCH_LIMIT	Maximum number of notices fetched per query. Currently: 50. Increase if notice volume grows.	No
PRIMARY_COLOR	AppColors.primary = Color(0xFFC8102E). Mahindra Red. Change if brand guidelines update.	No
MOODLE_COLOR	AppColors.moodle = Color(0xFF7C3AED). Purple for MU Website card. Cosmetic only.	No
FIRESTORE_REGION	Firebase project region: asia-south1 (Mumbai). Set during Firebase project creation. Cannot change without migration.	No
MIN_PASSWORD_LENGTH	Minimum password length enforced by Firebase Auth: 6 characters. Can be increased via Firebase Auth settings.	Yes

8 Quality of Service

This section describes the design decisions specifically made to address availability, security, performance, and monitoring requirements of the MU App.

8.1 Availability

The MU App targets 99.5% availability during academic hours (6:00 AM – 11:00 PM IST). Availability is primarily dependent on Firebase infrastructure, which provides a 99.95% SLA for Firestore and Authentication.

- Offline reads: Firestore local persistence is enabled by default in the `firebase_core` SDK. Cached notices and user profiles are available without internet.
- Graceful degradation: If Firestore is unreachable, `NoticesProvider` sets `_error` and displays a user-friendly message. Other app features (ERP, MU Website) continue functioning independently.
- WebView resilience: `ERPScreen` and `MoodleScreen` provide a Refresh button so users can retry failed portal loads without restarting the app.
- Session persistence: Firebase Auth persists sessions locally. Users remain logged in across app restarts and network interruptions.

8.2 Security and Authorization

Security is enforced at three levels: client-side (UI conditional rendering), router-level (GoRouter redirect guards), and server-side (Firestore security rules).

- Authentication enforcement: GoRouter redirect guard checks `AuthProvider.isLoggedIn` on every navigation. Unauthenticated access to `/home`, `/notices`, `/profile` redirects to `/login`.
- Role-based UI: `FloatingActionButton` for posting notices is only rendered when `role == 'faculty' or 'admin'`. Client-side check only — Firestore rules are the authoritative enforcement.
- Firestore security rules: `users` collection — read/write only by matching `uid`. `notices` collection — read by any authenticated user; write only by `faculty/admin` role verified via `get()` in rules.
- No secrets in client: No API keys are embedded in the Flutter app. Firebase configuration in `firebase_options.dart` contains only project identifiers (not secret keys).
- WebView cookie isolation: ERP and MU Website WebViews use separate cookie stores. ERP session cookies are not accessible to app JavaScript.
- Password security: Minimum 6 characters enforced by Firebase Auth. Future enhancement: enforce uppercase + digit + special character in `RegisterScreen` validator.

8.3 Load and Performance Implications

The following performance targets drive specific design decisions:

Metric	Target	Design Decision
--------	--------	-----------------

App cold start	< 3 seconds on 4G	Firestore initialized asynchronously in main(). Firestore cache serves Home screen data immediately on restart.
Notices fetch latency	< 2 seconds on 4G	Firestore query limited to 50 documents with orderBy index. Client-side filtering avoids complex server queries.
WebView first paint	< 4 seconds on 4G	LinearProgressIndicator shown immediately. JavaScript injection deferred to onPageFinished.
APK size	< 30 MB	No large asset bundles. No AI libraries. Minimal pub dependencies.
Concurrent users	5,000+ Firestore listeners	Firestore scales automatically. Each user has one active listener on the notices collection.
Firestore reads/day	< 50,000 (Spark limit)	Notices limited to 50 per fetch. Caching reduces re-fetches. Provider pattern prevents duplicate reads.

8.4 Monitoring and Control

In the MVP release, monitoring is handled through Firebase console tools. No custom monitoring infrastructure is implemented.

- Firebase Authentication console: Tracks registered users, sign-in activity, and authentication errors. Accessible at console.firebase.google.com.
- Firestore usage dashboard: Monitors daily reads, writes, and storage consumption against Spark plan limits (50,000 reads/day, 20,000 writes/day).
- Flutter debug mode: flutter run in debug mode provides hot reload, widget inspector, and console logging for development monitoring.
- Error logging: All caught exceptions are printed to console via print() in Provider classes. Future enhancement: integrate Firebase Crashlytics for production crash reporting.
- Controllable processes: No background daemons or message handlers in MVP. Firebase Messaging (FCM) is initialized but not actively used for scheduled notifications in v1.0.
- Future monitoring: Firebase Performance Monitoring SDK can be added in v2.0 to track screen render times, HTTP request latency, and app start time automatically.